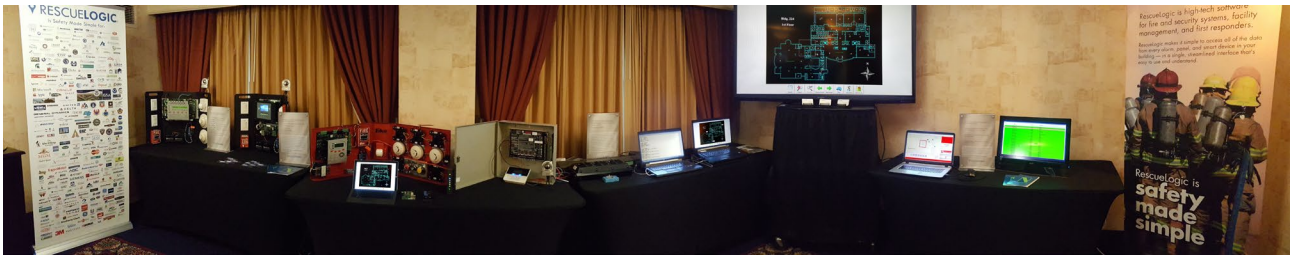
CADGRAPHICS INCORPORATED / RESCUELOGIC

One architecture. Many manufacturers. Decades of continuity.

How Cadgraphics developed a common architecture for understanding, organizing, and verifying fire-alarm information across manufacturers and generations of equipment.

Engineering history with firsthand observations • October 2026



2016 Marriott demonstration — original panoramic photograph taken in 2016.

The common foundation behind the products

In the beginning, my motivation was practical. I wanted to make software that could serve as many customers and manufacturers as possible without having to rebuild it each time. Finding the information that different fire alarm systems had in common was the most efficient way to accomplish that.

Only over time did I come to appreciate the larger consequence. The same architecture that made my software adaptable also helped customers preserve their knowledge of their buildings as manufacturers, equipment, and technologies changed.

What began as a practical business decision has become something I believe is worth sharing.

Looking back, I think I may have been asking myself a very simple question: What does everybody know? Not the technical details of every fire alarm panel, but the basic questions people naturally ask when something happens: What happened? Where did it happen? What should I do?

My early fire-alarm software work, beginning in the 1980s, focused on recurring questions: which device was reporting, what kind of device it was, where it belonged, and what condition it reported. Manufacturers communicated differently, but those questions remained much the same. Cadgraphics connected the answers to maintained customer records.

Over the years, we developed different ways to receive and interpret messages from fire alarm equipment. The methods became easier to configure, but the customer information they supplied remained consistent. Manufacturers could also offer the software under different product names; branding and panel connections changed without requiring a new information model.

As the years went on, I increasingly wanted to give the people who knew the building the methods they needed to maintain accurate information without depending on me. Seeing their appreciation for what the software allowed them to accomplish became one of the greatest rewards.

The common information behind every report

Fire alarm equipment does not use one universal message format. But the reports can be interpreted in recurring information roles. Those roles are the common structure RescueLogic used; this does not mean every panel transmits every field or uses the same terminology. [1–3]

This system reports that this device, of this type, at this location, has changed to this state.

SYSTEM	DEVICE	TYPE	LOCATION	STATE

The five roles are a way to read a report, not five fields guaranteed to arrive in every message. A reporting system identifies a point; device type and location may be supplied by the message or by existing, separately maintained site records. The reported condition is what the source says about the device or system. The report does not, by itself, establish the actual physical condition or its cause. Missing or unverified details must remain distinguishable from established ones. [1, 2, 6]

From reports to maintained information

RescueLogic interprets incoming reports and connects them to customer records. The original received messages, interpretation rules, device identities, descriptions, drawings, and event histories each have distinct purposes. Keeping them separate allows a technician to compare sources and investigate differences rather than overwrite one record with another. [4]

What a record can—and cannot—establish

A panel report, a database entry, and a floor-plan symbol may agree about a device. Their agreement is useful evidence of consistent records, but it does not by itself confirm the equipment's physical location or actual condition. That requires an appropriate check against the installed equipment, configuration, and field findings. [4]

The common model does not establish the truth of every report. It gives people a consistent way to identify, compare, investigate, and verify what was reported.

Field experience and its relevance

The same architecture was used in installations of very different sizes and configurations. These experiences show particular methods for preserving information through equipment replacement, correcting interpretation, and testing results. They are examples of work performed, not a blanket certification of every device at those installations.

Field experience	What the experience demonstrates
A campus: about 40 buildings	Old and new panel networks were monitored together for most of a roughly twenty-year transition. The migration didn’t have to happen all at once in a rip-and-replace upgrade.
Eleven buildings: about 20 miles	As field technicians tested devices across eleven buildings, we could replay their recorded messages at the central server and correct how RescueLogic interpreted them. We could then check the results without asking technicians to operate the same devices repeatedly. This made troubleshooting faster and reduced unnecessary field testing.
Federal headquarters: about 700 tested devices	During a major acceptance test, the security restrictions prevented us from adding files to the server. Fortunately, RescueLogic's installed Configuration Manager allowed us to correct how an uncommon supervisory message was interpreted. Approximately 700 devices were tested successfully without installing new software.

The site experiences are described in more detail in Sections 01, 02, and 06.

What the examples establish

Across these different installations, and many other examples, the same architecture allowed information to remain useful while equipment, communication methods, and operating conditions changed. The ability to examine, correct, and verify that information was as important as displaying it.

Start with the person who uses the information

Do they understand it? To keep that understanding accurate over time, we must preserve local knowledge and familiar terminology, make differences visible, and teach people how to check and correct their records.

RescueLogic was not meant to decide what people should know about a building. It gave qualified people tools to compare what their equipment reported with their maintained records, determine what needed investigation, and preserve verified corrections. The reports establish what was received; the customer and responsible technicians establish what has been checked.

Executive summary | Engineering lessons

RescueLogic is a Cadgraphics life-safety information and integration platform. Its historical architecture connected reports from different fire alarm systems to a common set of customer installation records. Methods of communication, interpretation, branding, and presentation changed over time; the recurring information roles remained useful.

What the engineering history demonstrates

A common foundation. Manufacturer reports could be interpreted and related to the same kinds of customer device, location, and condition records.

Adaptable translation. The 2010 Regex Script Panel made many message interpretation rules configurable rather than requiring a new compiled interpreter for each ordinary format.

Continuity across upgrades. Database upgrade methods helped carry device identifiers, descriptions, drawing placements, and event information forward as systems changed.

Repeatable investigation. Preserved source messages and test scripts made it possible to check interpretation changes against recorded input and expected results.

Maintainability by others. The methods could be taught to qualified owners, technicians, and service providers; accuracy still depended on checking the installation.

A new exchange direction. SAFER is a later proposed event format, and Game Plan extends the reuse principle toward additional workflows. Neither is presented as part of the original software releases.

What remains to be demonstrated

The installation examples do not establish capacity, cybersecurity, acceptance, or reliability for every proposed future deployment. New applications and integrations require their own defined scope, controlled testing, operating responsibilities, and evidence. Section 07 outlines that evaluation approach.

Historical scope and certification

This account combines preserved project records with firsthand recollections, identified later in the report. UL issued Recognized Component certification for RescueLogic software 5.0.0 in June 2017; that historical recognition does not certify new software versions or a complete installed system.

The enduring purpose is to help people preserve an accurate understanding of their installations, even when equipment, contractors, computers, and software interfaces change.

01 Many manufacturers, one evolving architecture

Over the years, many companies offered our software under different names. The names and panel connections changed, but the underlying RescueLogic software and its customer information model remained largely the same.

The core installation model was shared from the early CADGRAPHICS years. Different manufacturer translation interfaces fed the same customer records. By 2010 the Script Panel made most translation rules configurable; the small Mfg.INI file handled company presentation separately. The names below identify uses of the same software and the interface work that connected it to different panels.

Manufacturer / partner	Product name or connection	What was different
Cadgraphics	CADGRAPHICS; RescueLogic	CADGRAPHICS and RescueLogic shared a consistent customer data model. Translation interfaces varied; Mfg.INI supplied company identity separately.
Fenwal / Kidde-Fenwal	Early CADGRAPHICS interfaces	The earliest panel connections showed how much field support and practical installation problems mattered.
Fike	Precise Vision	RescueLogic supplied as Precise Vision, with a Fike-specific interface and configuration. Special features were developed for Fike, but the underlying software and customer information model remained the same.
Siemens	InSight Light	RescueLogic presented as InSight Light, rather than a separate Siemens graphics program.
Faraday (later Siemens)	MPC-NET / MPCNW	Dedicated channel and control engineering with Faraday illustrated the demands of interactive panel functions.
Hochiki	EMS Graphix	RescueLogic supplied under the EMS Graphix name, using the common software.
Mircom / Secutron	CG2000; MR-GRID-II	The same Cadgraphics software appeared as CG2000 and MR-GRID-II, with manufacturer-specific translation interfaces.
Advanced Fire Systems	Ad-VIEW	Cadgraphics software presented as Ad-VIEW, developed with Advanced engineers; an Advanced dealer later installed RescueLogic in prisons in 2025.
Alertus Technologies	RescueLogic under the Alertus name	The same RescueLogic software appeared under the Alertus name. Work with Blake Robertson on message boundaries and timing helped refine the Script Panel.
Global Fire Suppression	Global Vision	RescueLogic supplied under the Global Vision name, with manufacturer presentation distinct from program logic.
iLinkx LLC	iLinkx; GraphicLinkx	Cadgraphics graphics software appeared under the iLinkx and GraphicLinkx names.
Evax Systems	Tracker Graphics	Tracker Graphics was an Evax identity for the shared software; Harrington used the Tracker name with the same panel family.
Harrington Signal	Tracker Graphics	Tracker Graphics also appeared with Harrington.
Commercial Wireless Systems International	CWSite	RescueLogic appeared under the CWSite name, and was included in a project expansion at Duke University.
Affiliated Customer Service	NetVision 2020	NetVision 2020 was another name used for the common Cadgraphics graphics software.
FireLogic Indonesia	FireLogic	RescueLogic supplied under the FireLogic name for a regional channel.
Alarm Tech Solutions	First Responder	RescueLogic presented as First Responder, using the common graphics application.
Gamewell / Gamewell-FCI	600-series and later interfaces	Early dedicated interface and PC control work showed how closely panel behavior and communication could be coupled.

Manufacturer / partner	Product name or connection	What was different
Potter Electric Signal	XML panel interface	Discussions with Potter engineers about XML reporting showed another path toward structured event information. The XML interface has also been used successfully in more recent installations.
FCI / Fire Control Instruments	CADGRAPHICS price-book offering	An early manufacturer channel for CADGRAPHICS, around 1994 according to firsthand recollection.
Silent Knight	Dedicated interface; Gateway polling	Gateway polling worked on the bench but did not scale satisfactorily at a larger site; printer output proved more practical.
Bosch	GV4 (third-party)	Interactive exchanges and timing requirements beyond parsing printer reports.
Notifier	Script and importer interfaces	Importer work and later Regex Script configurations, including the 2016 Marriott demonstration.
Additional panel makers	Simplex; Edwards; others	Additional manufacturer-specific translations brought different reports into the common customer information model.

These product names are part of the same software's history. Dedicated interfaces and later Script Panel rules translated most later manufacturer reports into a consistent customer data model.

Fenwal and the first practical connections

The earliest CADGRAPHICS installations used Fenwal panels. They established an important lesson: panel reports could identify devices as events occurred and support the technicians responsible for the installation.

Fenwal's internal support people referred distributors to Cadgraphics when a graphics application was needed. Those referrals continued for decades and kept the engineering work close to field problems.

I especially remember Bill Garvey, who would say that he wasn't always sure adding graphics was the best idea, but that if a distributor was going to do it, they should call Dan Horon at Cadgraphics.

That recommendation reflected practical experience. A product name was not the point in those days. The important things were whether the software worked and whether someone else could maintain it.

The same principle carried into later projects: understand the installation, solve the immediate problem, and leave behind a method that other people can maintain.

Alertus: lessons in message boundaries and timing

Beginning in 2012, work with Alertus CTO Blake Robertson examined panel-script messages, embedded newlines, repeated events, message boundaries, and timing. Small differences in a character stream could turn a seemingly simple connection into a challenge in the field. Working through those problems helped refine the Script Panel.

The collaboration also brought RescueLogic to users under the Alertus name. Its lasting engineering lesson was the value of testing real input, retaining source text, and adjusting Script Panel rules without recompiling the application.

Product names and enduring installation records

The product name and the information about the building were always different things. Before 2010, manufacturer-specific translation interfaces already fed the common installation model. Later labeled editions used Mfg.INI for company names, contact details, splash screens, and logos while retaining the same RescueLogic program.

Cadgraphics continued developing database upgrades so that point identities, descriptions, and placements from older installations could move into later RescueLogic systems rather than being recreated whenever the computer, contractor, or version changed.

Lessons carried from one project to the next

Projects took different paths, and commercial outcomes varied. Their engineering value came from making a working connection, understanding its limitations, and reducing the effort needed for the next variation.

Over the years, what accumulated was principally methods, not just information. Each installation helped refine ways to receive messages, recognize their meaning, compare them with customer records, investigate differences, and verify corrections.

Most of those methods were straightforward. The value came from learning which ones worked best, making them easier to repeat, and giving the next technician a way to use them.

A consistent data backbone

The long-running installation model used Cadgraphics.mdb for configuration and CGLog.mdb for event history. Fike's Precise Vision used Vision.mdb as a configuration filename. Database names and schemas could evolve while the core customer relationships remained consistent. Upgrade routines and migration notes carried installation records forward as software versions changed.

The preserved database structure and upgrade history help protect that knowledge even when computers and screen graphics change. In the future, structured records may also help tools identify missing details and propose comparisons for qualified people to review.

One college campus, two generations of panels

At a college campus, RescueLogic monitored the older Gamewell 600 Series network and the newer Gamewell-FCI E3 systems side by side across a campus of about 40 buildings. Replacement proceeded over roughly twenty years, one panel at a time, while the campus continued to have a common view of its fire-alarm information.

I showed the university how to move panels to the new network. Only later did we discover that the original floor plans already in RescueLogic had not been used to match the new device identifiers with their symbols on the drawings. The lesson was about training as much as compatibility: a successful migration must include checking new device identifiers against existing floor-plan symbols and teaching the customer's technicians how to preserve the location information already established.

The same knowledge under different names

A fire alarm system contains more information than the names and locations of its devices. It also contains the rules for understanding messages, the relationships between panels and equipment, and the history of what has happened at the site.

The owner's view of the building remained the reference point. Manufacturer-specific translation brought different reports into the same records, where people could compare descriptions and investigate differences. The 2010 Script Panel made that translation easier to adjust; Mfg.INI changed the product's name and graphics. The customer's knowledge of the installation remained the common foundation.

Cadgraphics also continued improving the database and upgrade routines. Records from earlier installations could move into newer RescueLogic versions instead of being discarded when the label, computer, or manufacturer changed.

Even when an older installation was not fully documented, its existing records could provide a starting point. Missing details could be added as they were verified.

It was important to make the software robust to keep running. It was also important to preserve the meaning stored in the installation while the technology around it changed.

02 Improving translation around the common model

From dedicated code to adjustable rules

From the beginning, different fire-alarm panels reported events in different ways. A message from Notifier might look nothing like a message from Fike. But RescueLogic needed to answer the same basic questions for both: Which system? Which device? What kind of device? Where is it? What condition is it reporting?

Cadgraphics developed interfaces that translated those different reports into the same customer records.

In the early years, the instructions for communicating with a panel and interpreting its messages were often written together in the program. Over time, we separated those two jobs. By 2010, the Script Panel allowed many message interpretation rules to be changed on site.

Early interfaces sometimes asked questions

Some dedicated interfaces interpreted printer-port text directly. Others used interactive protocols: the host computer asked a panel for information, waited for an answer, and then selected the next question.

What a polling test taught about scale

The most demanding interfaces were polling systems that only answered prompts from our computer, which had to actively ask for every piece of an event message like in a game of charades: “Do you have anything to report?” Then, “Is it Panel 1?”, “Is it Panel 2?” followed by “Is it on Loop 1?”, “Is it on Loop 2?”

I built a Silent Knight Gateway polling interface that worked on the test bench. At a larger installation with approximately 700 devices, five panels, and additional remote annunciators, the polling approach did not scale as expected. Using the panel's existing printer output proved to be the better solution because it generated the whole sentence in one quick burst.

Bosch's third-party GV4 interface was a different case, but also required an ongoing conversation that could not be reduced to a finished event message.

As panel formats evolved, changing terms began moving into external lists. One list identified internal panel troubles; another classified Alarm, Trouble, and Normal. These lists could be maintained without modifying compiled program instructions.

The next step in translation combined words, positions, patterns, and lookups in one configurable interface: the Regex Script Panel. The customer information it populated was already common.

Configurable rules replaced repeated code changes

A list alone could not handle every format. Sometimes a word identified a field; sometimes meaning depended on position, surrounding characters, or a numeric code. Those variations needed field-configuration choices rather than a new program for each panel.

The Regex Script Panel brought the methods together. A regular expression, or Regex, is a rule for recognizing a pattern in text. Each field could use a list of known terms, match words and patterns, or find information by position. Source retained from February 2010 already shows the generic approach.

With the Script Panel, trained maintainers could change and test many manufacturer message interpretations without waiting for a new System Monitor build. The established customer model remained the destination; the translation rules became easier to adjust and verify.

I remember realizing that the Script Panel had replaced nearly every hard-coded message interpreter I had written. That didn’t eliminate special cases like the Bosch GV4 interface, but it removed the need to recode most ordinary complete-event message formats.

A test that could not wait for a software update

At a federal headquarters site about fifteen years ago, roughly thirty engineers and managers assembled to observe proper reporting from about 700 of some 7,000 devices. Only after arriving did I learn that the secured computer allowed no file transfers during the week of testing. Fortunately, its Configuration Manager and Script Panel tools were already installed.

During testing, an uncommon supervisory report failed to appear on the three System Watch computers being observed. The field team waited while I examined the message, adjusted its interpretation rules, and checked the results using the Configuration Manager and Script Panel tools already installed on the server. The approximately 700 devices exercised during the test ultimately reported as expected. The group ended the week confident that the remaining devices would work, and subsequent years of operation reinforced that confidence.

A common destination for different reports

Panels use different serial and network connections, and some report a completed event while others require an interactive conversation. The pathway that delivers characters is distinct from the rules that recognize their meaning.

Two ways to recover useful information

The Script Panel extracts available information from a complete report. When an event code needs context, a configured lookup supplies its known interpretation for that panel and installation. Semantic extraction asks what the source reported; normalization makes known terms comparable. Neither establishes an unchecked physical location.

Layer	Purpose / practical use
Communication	Receive panel or gateway traffic over the configured serial or network connection.
Direct communication log	Keep original received buffers and PC timestamps for diagnosis and replay.
Message framing	Separate complete messages; retain incomplete trailing text for the next read.
Semantic extraction	Identify the reported condition, device ID, type, location, and available time.
Normalization	Map panel-specific terms or codes to common meanings using configured rules.
Configuration	Relate the report to its site, panel, device, owner description, and drawing.
History	Retain interpreted activity for review and maintenance, separately from the original log.

The matchers recover fields independently, so missing optional text need not discard everything useful. Message framing and completeness checks still determine whether the material can be processed as an event.

RescueLogic writes each newly received character buffer once to its direct communication log, including incomplete messages. Every historical log uses the same markup-like delimiters: <!LogData> encloses the received chunk, with its PC time recorded in <!LogTime> tags. Trailing incomplete characters are retained and prepended to the next read for interpretation, but are not logged again as new arrivals. The original chunks preserve significant CR and LF sequences and help reveal gaps between portions of a message.

That matters because the characters on a terminal are not always the whole story. One fire-alarm system started its messages with a line feed and ended them with a carriage return. Another sent a NUL character every minute as a heartbeat. NUL normally did not appear on the display, so at first we missed it. Inspecting the original input made that otherwise invisible communication behavior understandable.

THE CHAIN OF MEANING

Element	Role in RescueLogic
Panel	Reports a condition using its own message format
Direct log	Preserves the original received characters
Interface	Extracts the reported condition and device information
Database	Identifies the device and maintains its recorded information
Drawing	Relates the device to a floor plan and its location
History	Records reported events for review, printing, and maintenance

Each part of RescueLogic has a job. When equipment, software, or a description changes, the relationships between received information, devices, and locations must be checked. A working display does not guarantee that all of its underlying information remains correct.

More than displaying what the panel said

RescueLogic developed into an analytical life-safety information system. Panel messages, installation records, floor plans, and event history offered several ways to compare information, identify differences, and improve the maintained record.

What “learning” meant as a system grew

As systems grew, RescueLogic retained information about previously unknown points and organized the reports for review. This became known as learning from the installation: useful reported information could be saved even before every detail had been checked in the field.

Here, learning means retaining and organizing reported information. It does not mean that a new entry has been checked against the installed equipment. A reported address, type, or description may still need to be compared with the panel configuration, drawings, and what is actually in the building.

Correcting information by comparing sources

Retaining reports about unknown devices lets people investigate differences instead of accepting the first description as final. A better owner description can be entered, a panel identity checked, or the discrepancy referred to the technician responsible for panel programming. The purpose is to improve the maintained information deliberately.

The useful cycle was to receive information, compare it with what was known, investigate the differences, and add verified knowledge to the maintained record. The system could become better documented over time instead of forcing the next person to begin again.

A simple example of the method

A new smoke detector or elevator-recall point may begin reporting before anyone has placed its symbol on a drawing. In one installation, devices were added for elevator recall by floor following fire-department requirements. RescueLogic could recognize those newly reported points without requiring a completed floor-plan update first.

When a previously unknown point reports, RescueLogic adds its available ID, type, and panel-supplied location description to the database and places the point on the Unplaced Device List. When a technician places its symbol on a floor plan or panel drawing, the name of that background can become the location shown on lists and printouts. The point is no longer unplaced, but its actual physical location still needs appropriate verification.

The Owner Location description can also be maintained separately from the panel's description. This gives the people responsible for the building a way to improve the wording they use without silently rewriting what the panel reported.

Comparing the original report, maintained records, and installed equipment can lead to a verified correction. Agreement between computer-generated views is useful, but it cannot replace checking the real installation. Testing and verifying the result preserves knowledge for the next person responsible for the building. [4]

Technical note: device identity, state, and time

One way to visualize the event model is through these three related expressions.

$$\mathbf{DEVICE = (ID, TYPE, LOCATION)}$$

$$\mathbf{STATE = Normal \mid Trouble \mid Active}$$

$$\mathbf{EVENT = \Delta STATE \times DEVICE \times TIME}$$

Conceptual teaching shorthand from What Is an Alarm Event? [1]. The expression illustrates the relationship between a device, its reported condition, and time; it is not an arithmetic formula or a literal SAFER field definition.

The point retains its identity within its reporting system and relevant site context. Type and location describe that point; those descriptions may be corrected without necessarily making it a different point. A change of operating condition is distinct from a change to its documentation. [4–6, 8]

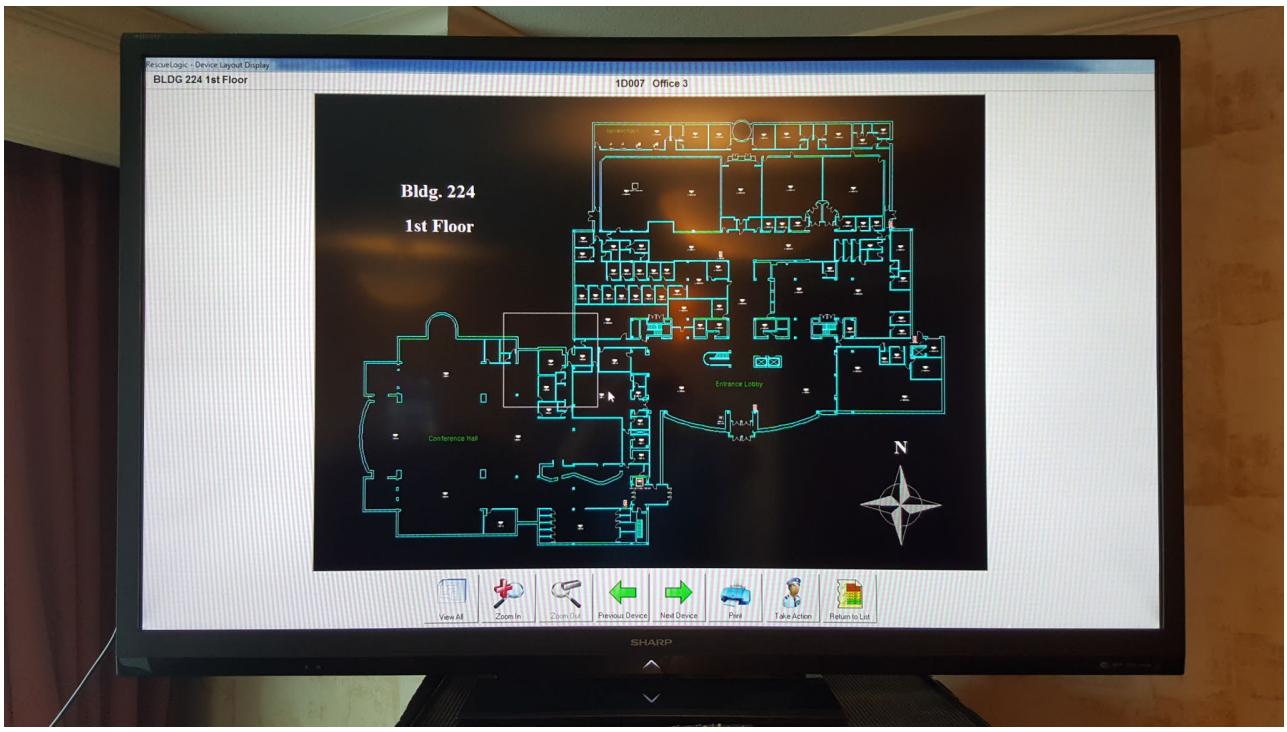
The three teaching categories above are not SAFER's six STATUS reporting codes or RescueLogic's separate Base and Logical State vocabularies. A received report may describe a new change, a restoration, a repeated condition, or a status observation; receipt alone does not establish a new physical transition. [5, 6, 8]

03 Marriott, July 2016: a shared view

The July 2016 demonstration at the Salt Lake City Marriott City Center brought several manufacturer systems into a single RescueLogic environment. The retained configuration identifies Notifier and Gamewell-FCI Regex Script interfaces and a dedicated interface for a Fike Cheetah Xi.

Demonstration material	What we can learn from it
Photographs	How the panels and displays were arranged for the presentation.
System schematic	How the intended equipment and network connections fitted together.
Configuration database	How Notifier, Gamewell-FCI, and Fike were represented in one application.
Event-history records	217 Notifier and Gamewell-FCI-associated entries in the archived review.

The archived review contains example Notifier and Gamewell-FCI-associated event-history records and also identifies the Fike interface. Additional equipment was operating but not exercised during the presentation; the Fike panel is used regularly in the Cadgraphics test lab.



A RescueLogic floor-plan display from the Marriott documentation.

What the demonstration illustrated

Different panels contributed to one understandable operational picture. Dedicated and configurable interfaces coexisted in the demonstration, both feeding the same installation model. The visible graphics brought that common information into a form people could use.

04 What continued as the industry changed

Manufacturers merged, products changed names, and some developed their own graphics offerings. Through those changes, Cadgraphics retained a common customer information model. Translation methods and software versions evolved around it; company presentation through Mfg.INI remained a separate concern.

What changed, and what stayed consistent

Period / approach	What changed	What remained consistent
Early CADGRAPHICS	Manufacturer-specific translation, often written directly into interface code.	Core customer records and the relationships among systems, devices, locations, and conditions.
Interactive protocols	Polling and successive node, loop, and device queries; specialized exchanges and timing.	The same need to identify the reported point and understand its condition.
Printer-output interfaces	Receive a report assembled by the panel rather than reconstruct it through repeated questions.	The common interpretation target and customer records.
By 2010: Script Panel	Our lists, patterns, positions, and lookups made most translation rules configurable.	The established customer information architecture.
Manufacturer identities	In the later labeled editions, Mfg.INI supplied company names, contact details, splash screens, and logos.	The same software, with branding separate from panel translation.
Upgrades and newer tools	Software versions, database schemas, and representations continued to evolve.	The meaning of installation records and the need to verify their relationships.

These approaches overlapped. Interactive interfaces and printer-output interpretation could coexist; manufacturer identity was a presentation choice, not a separate stage in the customer data model.

Modernizing the presentation without losing the information

Over the years, RescueLogic's screens changed less than the technology around them. I generally added or changed features when customers identified a need, rather than redesigning familiar screens simply to make them look newer.

That approach preserved established workflows, but it also meant that the appearance of the software could seem dated to someone seeing it for the first time.

The distinction matters. A screen's appearance, its usefulness to the person operating it, and the architecture that maintains its information are three different things.

Modern interfaces can make information easier to navigate and use on newer devices. Their usefulness should be evaluated with the people who depend on them, including improvements those users may not have thought to request.

The goal is not to preserve every old screen. It is to preserve what works, improve what needs improvement, and verify that the information remains accurate as its presentation changes.

Interoperability required distinct responsibilities

Some systems required interactive control, others dedicated data transfers, and others only the messages already generated by the panel. Certification, service arrangements, and network design also belonged to the larger installation. Configurable parsing simplified many interfaces without eliminating those other responsibilities.

Different names, the same application

In the post-2010 labeled editions, Mfg.INI carried the manufacturer name, contact information, splash screen, and logo. The executable software was the same. Before that, the translation interface varied while the core customer data remained consistent. Branding, translation, and installation knowledge had different jobs.

The continuing achievement was maintaining the customer's information while improving how different manufacturers' reports could enter and use it.

Turning the Architecture Outward

Originally, the challenge was to make one software application work with many manufacturers. We accomplished that by finding the information the systems had in common and translating their different reports into the same customer records. We traced their common language.

We started with the questions people had in common, discovered the information the systems had in common, and built a practical connection between the two.

Now we can apply the same principle in the other direction. Rather than building a separate body of information for every application, we can use the maintained installation records to support different interfaces for owners, designers, installers, inspectors, technicians, and responders.

The underlying information need not be recreated every time a different person needs to use it.

Over decades, manufacturers, equipment, contractors, and software interfaces have changed. The owner's responsibility for understanding the building has continued. Our opportunity now is to give owners and their authorized people better ways to preserve, compare, verify, and correct the information about their installations.

The goal is not simply to collect more records. It is to help the people responsible for the building establish what is known, recognize what remains uncertain, and keep their information accurate as things change.

Many manufacturers came into one model.

Now one model can open outward into many human purposes.

05 SAFER: carrying meaning forward

RescueLogic had long translated manufacturer reports into a common customer model. SAFER carries that principle beyond one application by giving reported meaning an explicit form that other software can interpret. [5, 9, 10]

The Security And Fire Emergency Reports (SAFER) Protocol proposes a structured JSON payload for reported events. It is later work, not a feature attributed to early CADGRAPHICS releases. The common information came first; the shared exchange format came later.

The recurring roles are reporting system, point identity, device type, location, and reported condition. Some facts come directly from a message; others require known source or site context. Missing facts must remain distinguishable from established information. [1, 2, 6]

SAFER standardizes the event payload, not the life-safety system. Communication pathways, supervision, equipment, certification, and response procedures remain separate responsibilities. The protocol is not tied to RescueLogic or Game Plan as its implementation. [5, 7, 10]

SAFER Protocol v1.1 is an October 2026 specification under Apache License 2.0. SAFER Common Vocabulary explains the terms; its proposed future changes do not amend the v1.1 payload. [5, 6]

The companion README introduces SAFER with FormattedOutput, a demonstration of RescueLogic displaying fire-alarm activity and formatting the same activity as SAFER JSON. [11]

SAFER publication and review site: https://github.com/rescuelogic/SAFER

Most companion publications are intended for this repository. Its README identifies published materials and revisions.

The mirror effect: shared meaning, different views

A panel message, database record, floor-plan symbol, and SAFER report can describe the same point and reported event. Their layouts differ; their shared meaning should agree. [1–3]

The Grammar of a Fire Alarm System explains the information roles; The Ordered Fire Alarm System follows them into comparison, verification, correction, and handoff. [3, 4]

A proposed path into other applications

SAFER JSON offers a possible exchange boundary for partner APIs, cloud services, analytics stores, or building-management applications. A pilot test should choose one receiving system and verify that site identity, device identity, source context, and reported meaning survive the handoff. Connectors, access controls, delivery behavior, and support would be designed and tested for the partner's environment. [5, 6, 10]

A bridge to Game Plan

Game Plan extends the same principle toward

Estimate → Design → Submittal → Installation → Testing → Maintenance

The goal is simple: reuse maintained device information rather than recreate it at every handoff.

The PDF/JavaScript demonstrations and existing web-based examples show working functionality. Complementary web tools and broader project workflows remain in development and will be demonstrated separately as they become available. [10]

The partner opportunity is to connect useful information without making the customer rediscover the building.

06 Why the information must remain useful

A working screen can be reassuring while the records beneath it become outdated. RescueLogic was developed to help people discover and correct those differences, not merely acknowledge displayed alarms.

The opportunity beyond a graphical annunciator

NFPA 72 establishes requirements for fire-alarm and signaling systems. RescueLogic also addressed an opportunity beyond those required functions: using received events to help keep installation records understandable and accurate.

For large facilities, engineers often specified graphical user interfaces. Such specifications tended to emphasize what appeared on the screen rather than how device descriptions, locations, and message interpretations would be maintained and checked.

A computer could do much more than display alarms.

The engineering question was: **If we're going to use computers and modern networks with fire alarm systems, why not also use them to improve the information we have about the installation?**

RescueLogic was designed to interpret incoming messages, recognize previously unknown devices, compare reported information with existing records, and give qualified people the tools to investigate differences and maintain accurate descriptions and locations.

These analytical and documentation capabilities extend beyond what NFPA 72 specifically requires. They are a practical complement to the code's objectives: better information helps qualified people understand and maintain their systems.

The 2019 edition of NFPA 72 introduced provisions for Building System Information Units (BSIUs), recognizing certain uses of computer-based information systems. Those provisions distinguish computer hardware requirements from additional requirements that apply when a BSIU provides fire alarm controls.

NFPA 72 also permits performance-based communication technologies under defined conditions. These provisions do not remove the need to satisfy applicable requirements for equipment, software, supervision, listing, and approval.

On June 16, 2017, UL issued a UL 864 Recognized Component certificate for RescueLogic software 5.0.0. That historical recognition applies to the identified software component and its conditions of use, not to a complete installed fire-alarm system or a new release. The certificate is reproduced in the appendix.

RescueLogic was designed to make reports from fire-alarm equipment more useful to the people responsible for the building.

A computer is a tool, not just an annunciator

A graphics workstation could easily be treated as a finished appliance: mount it, connect it, and expect it to operate. Yet the information describing a building needs attention long after installation.

The software organizes the information, but owners, installers, and service providers still need to agree on responsibility for checking device records and approving changes. Keeping the information accurate is a continuing task.

We don’t expect to turn every owner into a programmer. Our goal is to give qualified people who understand the building repeatable methods they can use and verify for themselves.

What a working screen leaves out

When people see only…	They may overlook…
A floor-plan screen	The device database that makes every symbol meaningful.
A panel connection	The protocol rules, field mappings and exception handling.
A working demonstration	Long-term maintenance, upgrades and test coverage.
A new PC or server	Database migration, communication settings and historical logs.
A network cable	Supervision, redundancy, ownership and change control.

Keeping equipment operational and keeping its descriptions and records accurate are related but distinct jobs. Both deserve an agreed process for changes and verification.

Teaching confidence through verification

Looking back, I sometimes demonstrated what RescueLogic could do without sufficiently explaining why it worked that way. The technician watching needed to see a repeatable method—not a trick that only the original programmer could perform.

A fire alarm system may remain in service for twenty years or longer. During that time, computers, contractors, technicians, and even the fire alarm equipment may change. RescueLogic was designed to preserve the information describing the installation and give the people who know the building practical ways to maintain it.

Even experienced technicians may hesitate to change a working computer. That caution is appropriate around critical systems, but routine corrections become manageable when a verified backup, required files, recovery procedure, and known test are available.

Confidence comes from checking a controlled change. Before a database update, preserve a reliable copy. Afterward, compare the expected records, dates, and resulting behavior against a known baseline. I never count a change as successful merely because the computer did not complain.

On a running installation, preserving a reliable copy means using an appropriate, consistent database-backup procedure and checking that the copy can be restored. A workstation may be replaceable, but the current installation data, scripts, drawings, license details, and communications settings must be recoverable together.

Recently a technician, uneasy about working on a database copy, sent me a picture of his files. I could see the installation database, its locking file, and a backup. Instead of telling him to trust the filenames, I asked him to switch File Explorer to Details view and check each file's name, date, time, and size. Those clues do not prove that a backup is restorable, but learning to inspect them is a first step toward a controlled change.

A useful test checks the entire information path: interpret a panel report, identify the intended device, locate it on the drawing, and check the resulting history. Differences give people something specific to investigate. Agreement between software views is one check; field verification establishes whether they describe the actual equipment and condition.

RescueLogic can open an archived panel message and replay it through Test Scripts without operating live fire-alarm equipment. After changing the interpretation rules, the technician can compare the expected and actual fields, then test additional archived messages for unintended changes.

Eleven buildings: test once, replay as needed

Another customer wanted to monitor eleven buildings spread across about twenty miles from one RescueLogic server. Starting at 6 a.m., a service team worked from site to site. When a test did not display correctly, the team paused while I used the saved message and Test Scripts to adjust and replay the interpretation, rather than repeatedly operating the field device.

At one building, the point identifiers had been entered with an unexpected letter prefix. The mismatch explained why the expected device was not recognized. Comparing the installation with service records also raised questions about device counts that needed separate field verification. By about 11 a.m., the customer was satisfied with the observed testing. The original input stayed the same; the matching rules were corrected and the results checked.

There is satisfaction in that process. For me personally, getting every piece of information into its proper place, and understanding why it belongs there, can be genuinely relaxing. Each successful check removes another uncertainty.

That discipline shaped RescueLogic's editable lists, Regex rules, database, event history, and drawing tools. Each offers another way for knowledgeable people to examine and verify information about the same installation.

A demonstration shows what the software can do. A repeatable handoff also provides versioned message examples, expected results, approved interpretation rules, backup and recovery instructions, and a method for checking later changes.

Preserving RescueLogic means passing along not just the software and its data, but the ability to verify that the information remains correct.

Class N: lessons about network responsibility

In June 2016, Cadgraphics published *Class N: Networks for Fire Alarm and Mass Notification Systems*, a guide to modern networks in fire-alarm systems.

The guide examines network supervision, redundant paths, shared networks, and the distinction between required fire-alarm pathways and supplemental monitoring.

The same concerns apply to RescueLogic: a change to a panel, network, or computer must be evaluated within the full installation, not just by whether messages still arrive.

This independent Cadgraphics guide reflects engineering work and standards participation; it does not imply NFPA endorsement of RescueLogic.

The preservation principle

To preserve RescueLogic, keep the software, configuration database, event history, original panel messages, interface rules, floor plans, change records, and a tested way to restore or migrate them. Just as important, keep examples that show how someone can check the result.

Preservation checklist | What the next technician needs

A qualified successor should be able to reproduce and verify an installation's interpretation, not merely reinstall the software. Each record needs a version, custodian, last review, and location of a reliable copy.

Record to retain	Verification / handoff expectation
Application and environment	Keep installation media, exact software build, licensed options, dependencies, operating-system assumptions, and a tested restore procedure.
Configuration and schema	Archive Cadgraphics.mdb or Vision.mdb, field definitions, panel/device identifiers, schema version, and each applied migration.
Event history	Retain CGLog.mdb or other event history separately from the raw communication log, including timestamps and source identifiers.
Interface logic	Save scripts, regex matchers, lookup lists, communication settings, message-boundary rules, and configuration export.
Raw messages and expected results	Keep direct communication-log chunks and their PC timestamp markers, including CR/LF sequences, incomplete messages, repeats, restorations, and sample expected outcomes.
Panel/source documentation	Maintain authorized panel programming exports, point lists, manufacturer protocol notes, and communications wiring/port details where available.
Drawings and locations	Preserve source floor plans, PDF backgrounds, placed-device mappings, owner descriptions, and records of field verification.
Changes and responsibility	Record what changed, who approved and tested it, the evidence reviewed, and who owns each future update.
Recovery and migration test	Periodically restore from a backup, replay representative messages, compare the point locations and history, and document pass/fail findings.
Change-verification record	For significant corrections, record the input or test identifier, expected panel/device/condition/location/history, actual result, approval, date, and any discrepancies resolved.

A replacement parser, database migration, or future AI-assisted tool should be compared against verified field examples. A pattern match is a useful observation, but the right device, condition, location, and resulting history still have to agree.

Learning by testing

When troubleshooting, I often say, "Let's find something that does work, watch it for certain, and go from there."

A communication path is like a chain. Every link has to work, so we examine them one at a time. We can use PuTTY to see what the fire alarm panel sends through its serial port to a 9-pin COM port, then check what the Moxa serial server receives and transmits, and finally test the connection across the site's network.

A useful demonstration begins by sending typed or pasted characters between two serial COM ports using PuTTY or another simple terminal. Next, we repeat the test using a serial-to-TCP/IP connection. Finally, we connect to the panel's actual serial output and observe what it sends.

Each successful test establishes another working link. When the next link fails, we've narrowed down where to look. Panel output settings, COM port configuration, drivers, and TCP/IP routing can all affect communication.

Technicians can use these readily available tools to identify problems and verify their corrections.

07 A practical way to evaluate broader use

The proposed next step is a bounded evaluation, not a commitment to a portfolio-wide rollout. Select representative buildings, panel generations, and message formats, plus one intended receiving workflow. Use retained input and known expected results so the work can be repeated by a team other than the original developer.

Agree on acceptance criteria and numeric thresholds before testing. Keep the informational evaluation separate from required fire-alarm functions and site-approved operating procedures.

Proposed evaluation areas and evidence

Area	What the evaluation should establish
Meaning and identity	Trace original input through extraction, normalization, device records, drawings, and the receiving application. Check site/panel/device identity, missing facts, repeated reports, restorations, and source context.
Integration and security	Define the chosen destination, field mapping, approved transfer method, access controls, site separation, retention, and audit needs. Identify which controls exist and which require development.
Capacity and recovery	Agree event rates, burst loads, latency and recovery targets. Measure loss, duplication, ordering, and behavior after disconnects or restarts. Establish capacity from these tests rather than from building count alone.
Operations and training	Have a trained successor replay messages, change and review a rule, restore a baseline, and verify a result. Retain versions, approvals, rollback steps, support ownership, and escalation instructions.
Value and deployment decision	Compare technician time, repeat site visits, onboarding effort, and recovery effort with an agreed baseline. Record costs and exceptions before estimating savings or recommending wider deployment.

Responsibilities to establish

A broader evaluation would identify who supplies representative data, who maintains the interpretation rules, who verifies installation records, and who is responsible for security, support, and approving changes. These responsibilities should be established before considering wider deployment.

Advance on demonstrated results, a transferable operating method, and clear responsibility for keeping each building's information accurate.

The architecture and the lessons that endure

From the early CADGRAPHICS years, the architecture followed the customer's need to understand devices, locations, conditions, and history—regardless of who supplied the equipment. Manufacturer-specific translation methods changed around that foundation. The Script Panel made most interpretation rules configurable; database upgrades preserved the meaning of installation records as the software evolved.

The software appeared under different names while the customer information remained common. Mfg.INI supplied company presentation; interfaces handled manufacturer communication and interpretation. SAFER proposes a shared exchange form, and Game Plan extends the principle toward project work. Other organizations can evaluate these methods for broader use without losing the enduring purpose: information that people can understand, test, correct, and carry forward.

The questions were always there: **What happened? Where did it happen? What should I do?** Our head start came from decades spent finding easier ways to answer them—and teaching others to do the same.

My goal was never just to put a better display on the wall. It was to give people the tools to understand their installation and keep that understanding accurate as it changed.

Historical sources and development account

Engineering publication: Dan Horon, Class N: Networks for Fire Alarm and Mass Notification Systems (Cadgraphics Incorporated, June 2016, ISBN 978-0692724125). The following reading guide identifies the companion SAFER, event-model, and information-maintenance publications cited in this history.

Historical and technical records: July 2016 Marriott photos, schematic, configuration review; Cadgraphics interface source projects, OEM installers, and database upgrade records; 2011 Potter XML protocol correspondence; 2012–14 Alertus engineering correspondence with CTO Blake Robertson; and Cadgraphics' Alertus Sales by Customer Detail report (invoices 2012–2020, exported October 18, 2026).

Standards participation: NFPA Class N task-group materials dated January 2017, related technical comments, and the 2015 NFPA contributor agreement. These records document involvement in standards discussions; the independent Cadgraphics publication does not speak for NFPA.

Author’s development account: October 2026 recollections of the early common data model, dedicated interfaces, Silent Knight polling, Bosch GV4, the 2010 Script Panel, Mfg.INI, original communication logging, database continuity, and teaching technicians to test one link at a time. Additional field recollections include the overlapping college campus networks, a federal-headquarters device test, an eleven-building test, newly reported elevator-recall points, and file-inspection coaching. Early dates and site details based on recollection remain approximate.

Companion publications | Reading guide

The publications below explain the model, its use, and its proposed exchange format at different levels of detail. Bracketed numbers in this history refer to these editions. Cadgraphics Incorporated / RescueLogic, October 2026.

Ref.	Publication / edition	How it supports this history
[1]	What Is an Alarm Event? One-page teaching paper	The core sentence, device/state distinction, and mirror effect. Start here for the shortest explanation.
[2]	Fire alarm panels speak the same language Manufacturer-message comparison	Side-by-side formats show recurring information roles; some details require source or site context.
[3]	The Grammar of a Fire Alarm System Dan Horon · educational companion	Plain-language questions behind source, identity, type, location, and reported condition; one point in many views.
[4]	The Ordered Fire Alarm System Dan Horon	Identify, compare, verify, correct, and maintain. Distinguishes agreement between records from field verification.
[5]	The SAFER Protocol Version 1.1	Reference for the proposed v1.1 JSON payload, required fields, reporting codes, and scope.
[6]	SAFER Common Vocabulary Companion discussion	Separates observation, condition, qualifier, classification, and response. Future design options do not amend v1.1.
[7]	SAFER Protocol FAQ Version 1.1	Explains scope, adoption, compatibility, testing, and the responsibilities outside the event payload.
[8]	A Minimal Event Information Model — Review Version Dan Horon · Revision 0.2	Condensed technical treatment of the five information roles, stable point context, and changing condition.
[9]	A Minimal Event Information Model — Full Technical Working Paper Dan Horon · Revision 0.2	Expanded treatment of the reported-event model, interpretation, report identity, and interoperability.
[10]	Information Infrastructure for Life Safety Dan Horon and Jeff Horon · v1.1	Architectural companion: transport versus meaning, a common event layer, and RescueLogic/Game Plan examples.
[11]	SAFER Protocol repository README Version 1.1 context	Navigation, publication roles, the FormattedOutput demonstration, and implementation/review guidance.

Publication and review site: github.com/rescuelogic/SAFER

Most companion publications are intended for the SAFER repository. Consult its README for published files and current revisions as they become available. The educational guides explain the model; the protocol defines the proposed v1.1 payload.

Appendix | Historical reference notes

These references help readers follow the engineering history to its underlying material. Some come from preserved software, correspondence, certificates, and demonstrations; others are firsthand recollections of design choices and field experience.

How to read these references

Preserved files and firsthand recollections answer different historical questions; dates based mainly on recollection remain approximate. The core customer model preceded the 2010 Script Panel. Dedicated translations, later configurable rules, and Mfg.INI presentation settings describe different parts of that continuing architecture.

Example	Surviving material or recollection	What it helps explain
Fike Precise Vision	Branded software and Vision.mdb configuration	The same customer information model under the Precise Vision name, including its Vision.mdb configuration filename.
Siemens InSight Light; other OEM suites	Installers, product profiles, and Mfg.INI presentation settings	Different product identities for the same software; manufacturer presentation separate from translation and customer data.
Faraday; Silent Knight; Bosch; Gamewell	Source projects and firsthand interface-engineering recollections	Dedicated and interactive translations fed the common model; configurable interpretation later simplified many interfaces.
Potter XML work	2011 protocol correspondence	Early interest in carrying event information as structured data.
Alertus work	2012–14 correspondence; subsequent licensing records	How message boundaries and event timing became real engineering lessons.
Fenwal and early FCI	Firsthand recollection and historical price-book material	The earliest distributor relationships and panel-interface work; early dates are approximate.
Marriott, 2016	Photographs, schematic, configuration review, and 217 archived event-history records	A practical multi-manufacturer demonstration; the archive distinguishes configured equipment from exercised events.
Database continuity	Cadgraphics.mdb, Vision.mdb, CGLog.mdb and upgrade routines	Core customer information remained consistent while schemas and versions evolved through controlled upgrades.
UL 864, 2017	Certificate for RescueLogic software version 5.0.0	Historical Recognized Component status, not a new-product or complete-system Listing.
SAFER 1.1	October 2026 Protocol v1.1 and event model companions [1, 5, 6, 9]	A later explicit exchange model for recurring information, not the origin of RescueLogic's common customer architecture.

Certification and standards records

The UL certificate belongs to RescueLogic software 5.0.0 as a Recognized Component in 2017, not to a complete installed system. The Class N publication and technical committee history show engineering participation without implying an NFPA endorsement of RescueLogic.

UL CERTIFICATE OF COMPLIANCE

CERTIFICATE OF COMPLIANCE

Certificate Number	20170616-S35839
Report Reference	S35839-20170614
Issue Date	2017-JUNE-16
Issued to:	Cadgraphics Incorporated 9218 Borad Street Boca Raton FL 33434
This is to certify that representative samples of	COMPONENT - CONTROL UNITS, SYSTEM ; COMPONENT - CONTROL UNIT ACCESSORIES, SYSTEM USR Component: RescueLogic software 5.0.0
	Have been investigated by UL in accordance with the Standard(s) indicated on this Certificate.
Standard(s) for Safety:	UL864 - Control Units and Accessories for Fire Alarm Systems
Additional Information:	See the UL Online Certifications Directory at www.ul.com/database for additional information

Only those products bearing the UL Certification Mark should be considered as being covered by UL's Certification and Follow-Up Service.

The UL Recognized Component Mark generally consists of the manufacturer's identification and catalog number, model number or other product designation as specified under "Marking" for the particular Recognition as published in the appropriate UL Directory. As a supplementary means of identifying products that have been produced under UL's Component Recognition Program, UL's Recognized Component Mark: , may be used in conjunction with the required Recognized Marks. The Recognized Component Mark is required when specified in the UL Directory preceding the recognitions or under "Markings" for the individual recognitions.

Recognized components are incomplete in certain constructional features or restricted in performance capabilities and are intended for use as components of complete equipment submitted for investigation rather than for direct separate installation in the field. The final acceptance of the component is dependent upon its installation and use in complete equipment submitted to UL LLC.

Look for the UL Certification Mark on the product.

Bruce Mahrenholz, Director North American Certification Program

UL LLC

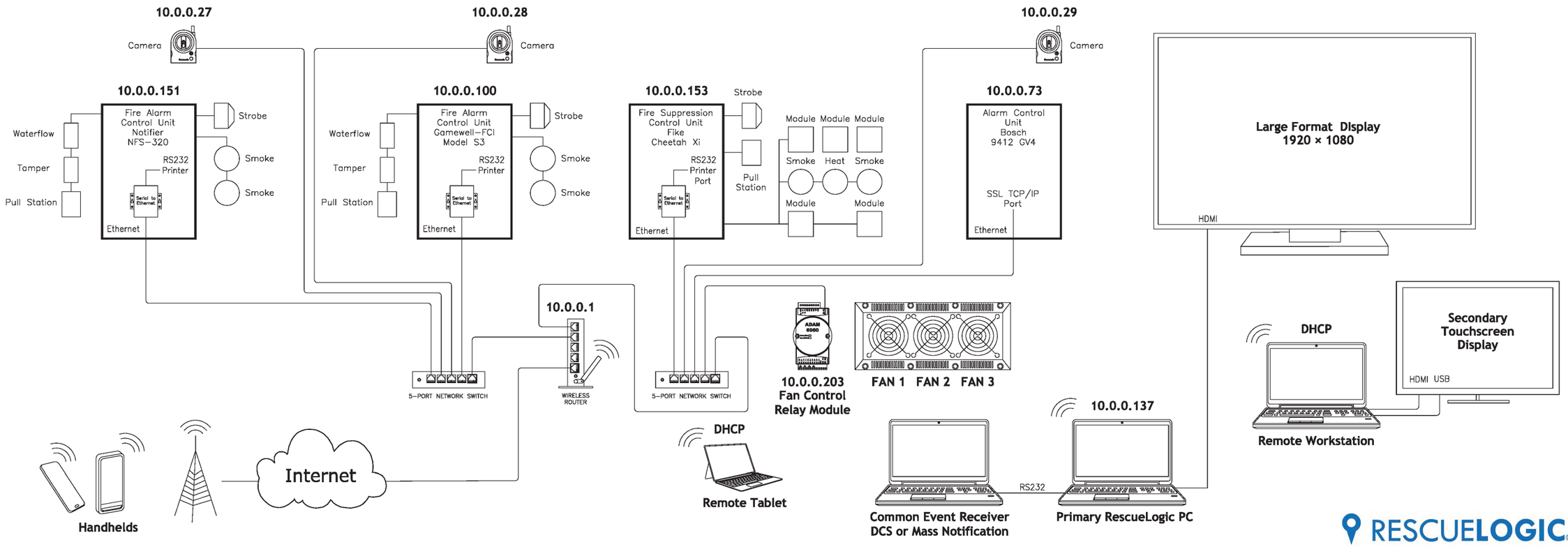
Any information and documentation involving UL Mark services are provided on behalf of UL LLC (UL) or any authorized licensee of UL. For questions, please contact a local UL Customer Service Representative at http://ul.com/about/ul/locations/

Page 1 of 1

UL 864 Recognized Component certification — RescueLogic software 5.0.0, June 16, 2017

MARRIOTT DEMONSTRATION





Original 2016 Marriott demonstration photograph and system schematic, preserved from the Cadgraphics case-study documents.